

JavaScript Basic

JavaScript 기초

JavaScript 기초 배우기

☑ JavaScript 기초



딱 쉬운 JavaScript!

이번 강의에서는 좀 더 쉽고, 초보자도 활용 할 수 있는 JavaScript 강의를 준비했습니다.

☑ JavaScript 기초 배우기



쉽게 배우고, 쉽게 이해해요.

제 1장 JavaScript 기초 배우기에서는 JavaScript가 무엇인지,
그리고 변수, 출력방식, 데이터타입, property, methods등 script의 기본이 되는
내용에 대해 공부해 봅니다.

✔ JavaScript 기초 다지기



쉽게 배우고, 사용법을 익혀요.

제 2장 JavaScript 기초 다지기에서는 JavaScript의 기본 문법인 연산과
조건문 반복문에 대해 공부하고 실습합니다.

JavaScript 소개

📌 JavaScript란 무엇일까요?



Hello JavaScript!

자바스크립트(JavaScript)는 객체(object) 기반의 스크립트 언어입니다.

HTML로는 웹의 내용을 작성하고, CSS로는 웹을 디자인하며, 자바스크립트로는 웹의 동작을 구현할 수 있습니다.

자바스크립트는 주로 웹 브라우저에서 사용되나, Node.js와 같은 프레임워크를 사용하면 서버 측 프로그래밍에서도 사용할 수 있습니다.

현재 컴퓨터나 스마트폰 등에 포함된 대부분의 웹 브라우저에는 자바스크립트 인터프리터가 내장되어 있습니다.

📌 용어 해설

Node.js

Node.js는 확장성 있는 네트워크 애플리케이션 개발에 사용되는 소프트웨어 플랫폼

✔ JavaScript의 특징



- 자바스크립트는 객체 기반의 스크립트 언어입니다.
- 자바스크립트는 동적이며, 타입을 명시할 필요가 없는 인터프리터 언어입니다.
- 자바스크립트는 객체 지향형 프로그래밍과 함수형 프로그래밍을 모두 표현할 수 있습니다.

✍ 용어 해설

객체지향 프로그래밍이란?

처리와 관련된 데이터와 절차를 하나로 묶어 객체 단위로 관리하는 언어

장점으로는 코드 재사용과, 유지보수가 용이함
단점은 처리속도가 상대적으로 느리고, 설계 시 시간과 노력이 필요

☑ JavaScript로 할 수 있는 것들



- HTML을 조작하여 콘텐츠 추가, 제거, 변경 할 수 있다.
- Canvas와 WebGL API를 이용한 2D, 3D제작을 할 수 있다.
- 오디오나, 비디오를 브라우저에서 재생 또는 조작할 수 있다.
- 다양한 API를 이용해 DOM(Document Object Model)을 조작할 수 있다.

✍ 용어 해설

API

Application Programming Interface의 약어로 웹 코드를 작성한다면 많은 API를 사용할 수 있습니다. API는 애플리케이션과 기기가 원활하게 통신할 수 있도록 하며, 모든 접속을 표준화하여 범용 플러그처럼 작동하게 합니다.

☑ JavaScript로 만들 수 있는 것들



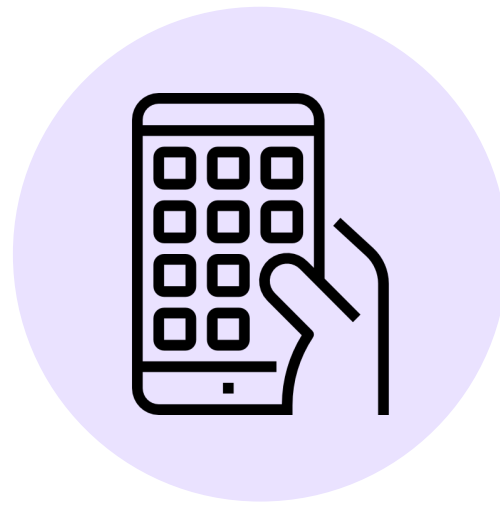
웹 사이트



서버 구축



웹 어플리케이션



모바일 앱



데스크 탑 앱

JavaScript 출력

☑ JavaScript의 다양한 화면 출력방법



```
alert('결과값을 확인해봐요!')
```

JavaScript에서 지원하는 다양한 출력형태에 대해 공부해봅니다.

또한 es6이후 버전부터 사용할 수 있는 backtick을 이용한 template 문법에 대해서 알아봅니다.

☑ Template literals



```
let age = 30;
let nickName = "Elice"
let hobby = "javascript";
let str = `저의 이름은 ${nickName}이며, ${age}살이고 ${hobby}를 좋아합니다.`;
console.log(str);    //저의 이름은 Elice이며, 30살이고 javascript를 좋아합니다.
```

- ES6 이전엔 표현식을 일반 문자열 안에 집어넣었습니다.
- 템플릿 리터럴에서는 아래와 같이 \$와 중괄호{}를 사용하여 표현식을 표기할 수 있습니다.
- 위 처럼 + 연산자로 문자열을 연결해주는 것보다 가독성이 더 좋습니다.
- 템플릿 리터럴은 작은따옴표(')나 큰따옴표(") 대신 백틱(`)(grave accent)로 감싸줍니다.

☑ Console.log()



```
// 브라우저 F12 개발자 모드에서 확인 가능  
console.log("개발자 모드에서 확인해요!");
```

- 데이터를 확인할 때 가장 많이 사용하는 출력형태 입니다.
- 브라우저의 검사도구를 이용해 console 탭을 열어 사용합니다.

✔ prompt()



```
prompt('너의 이름은', '')
```

이 페이지 내용:

너의 이름은

확인

취소

- prompt는 사용자에게 입력을 받을 수 있는 출력형태 입니다.
- 키워드를 대입해, 사용자의 입력 내용을 변수 값에 대입 할 수 있습니다.

☑ confirm()

```
confirm('확인하시겠습니까?')  
//확인 true  
//취소 false
```

이 페이지 내용:
확인하시겠습니까?

확인

취소

- confirm은 boolean을 반환합니다.
- 둘 중 하나를 선택하여 확인을 누르게 되면 true값, 취소를 누르면 false값을 반환합니다.

✍ 용어 해설

Boolean이란?

Data type중 하나로, 다음 강좌 data type에서 자세히 배웁니다. true, false 값을 반환합니다.

✔ alert()



```
alert('hello elice')
```

이 페이지 내용:

hello elice

확인

- 브라우저에서 제공되는 메시지박스를 이용해서 출력합니다. Test용으로 사용합니다.

☑ document.write()



```
document.write('hello javascript')
```

Browser

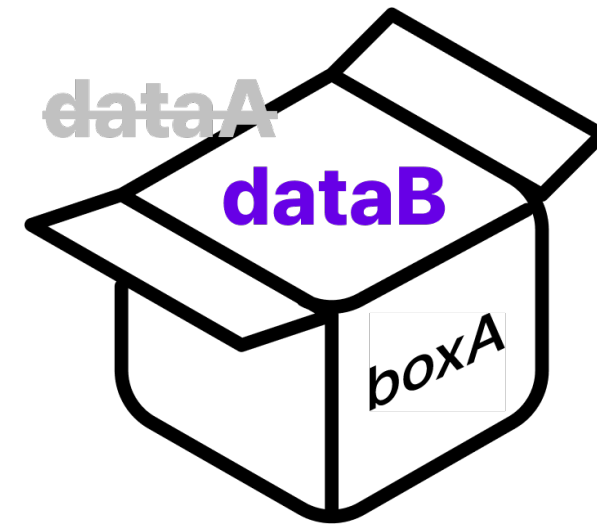
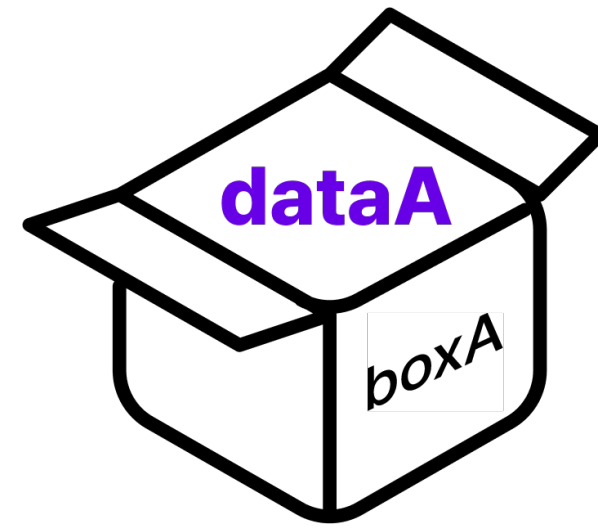
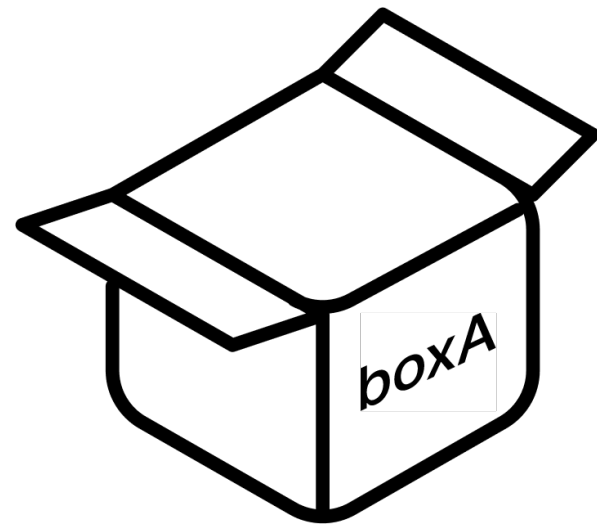
hello javascript

- 말 그대로 괄호 안에 들어가 있는 것을 페이지에 "쓰는(출력하는)" 함수입니다.
- HTML 문서에 표시되어 출력됩니다.

변수 익히기

변수란 무엇일까?

☑ 변수의 정의



빈 상자에 이름표를 붙이고, 데이터를 1개만 담아요!

- 변수Variable는 데이터를 담는 빈 상자입니다.
- 빈 상자에 이름표를 붙이는 것을 변수의 선언이라고 합니다.
- 상자의 크기는 모두 동일합니다.
- 이름을 통해 데이터를 사용할 수 있습니다. (재활용=재할당)
- 데이터가 재할당되면 기존에 있던 데이터는 사라집니다.



☑ 변수의 선언-키워드

let

const

var

변수를 선언할 때 사용하는 3개의 키워드

- 변수를 선언 한다. 라는 것은 변수명과 매핑하고자 하는 데이터의 메모리 주소를 저장할 또 하나의 메모리공간을 확보하는 것입니다.
- 변수를 사용하기 위해서는 꼭 선언이 필요합니다. 자바스크립트에서는 var, let, const 키워드를 사용합니다.
- let, const는 ES6부터 도입된 개념이며 그전에는 var 만을 사용해서 변수를 선언했습니다.

✍ 용어 해설

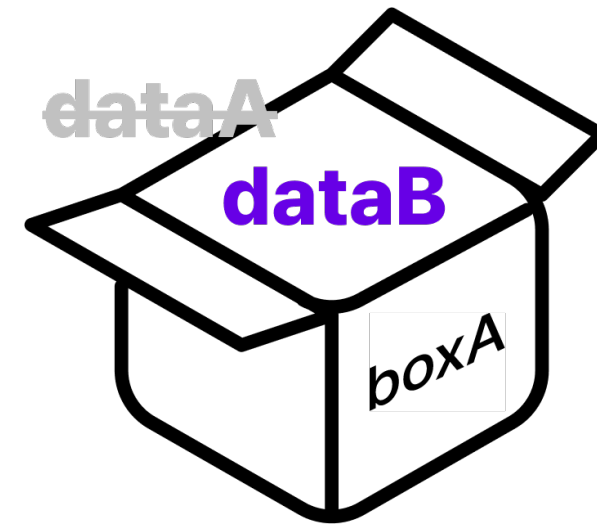
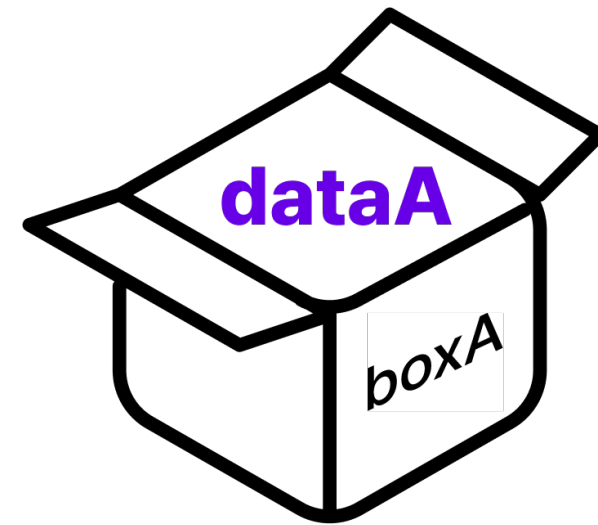
ES6?

ES란, ECMAScript의 약자이며 자바스크립트의 표준, 규격을 나타내는 용어이다.

☑ 변수의 선언-let

Block scope

let



데이터의 할당이 변경 될 수 있을 때 사용

- var 와는 다르게 let 키워드는 변수의 재 선언이 불가능합니다.
- let 키워드는 데이터의 재할당이 가능합니다.

🖋 용어 해설

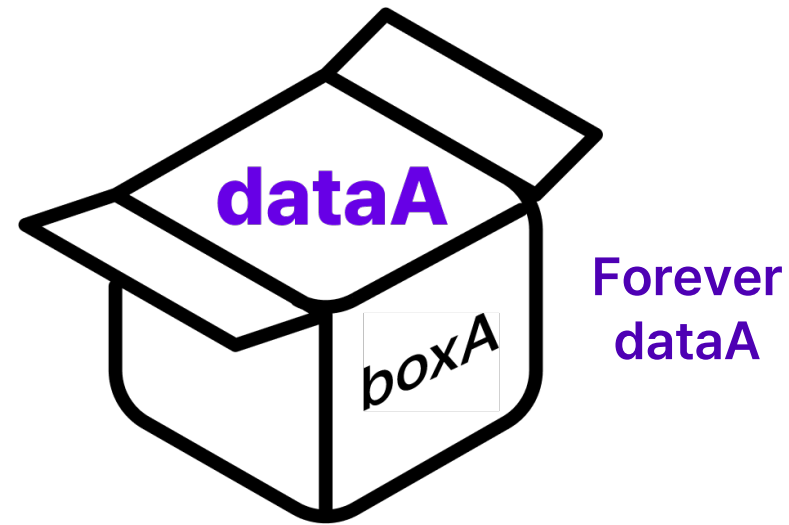
ES6?

ES란, ECMAScript의 약자이며 자바스크립트의 표준, 규격을 나타내는 용어이다.

☑ 변수의 선언-const

Block scope

const



데이터의 할당이 변경되지 않을 때 사용

- var와는 다르게 let 키워드는 변수의 재 선언이 불가하다.
- const 키워드는 데이터의 재할당이 불가하다.
- const가 let과 다른 점이 있다면, 반드시 선언과 초기화를 동시에 진행되어야 한다는 점이다.

✍ 용어 해설

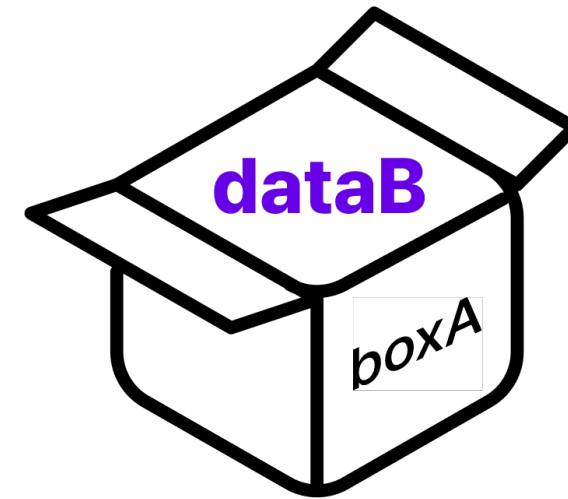
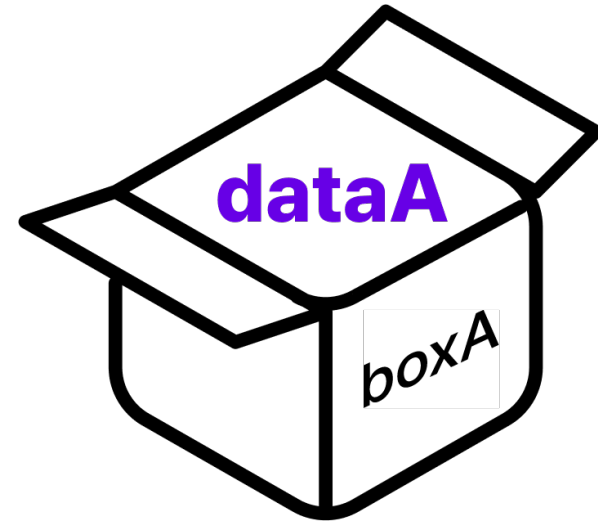
상수(constant)

재할당은 변수에 저장된 값을 다른 값으로 변경하는 것으로, 만약 변경할 수 없는 값이라면 상수로 설정

☑ 변수의 선언-var

function scope

var



ES6 이전 버전에서 사용

- 변수 중복 선언이 가능하여 예기치 못한 값을 반환할 수 있다.
- 함수 레벨 스코프로 인해 함수 외부에서 선언한 변수는 모두 전역 변수가 된다.
- 변수 선언문 이전에 변수를 참조하면 언제나 undefined를 반환한다.

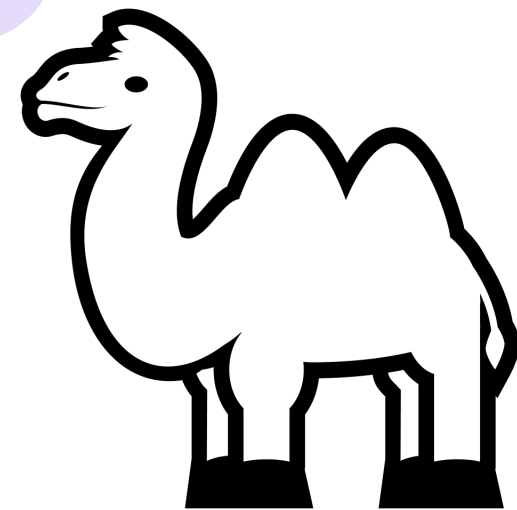


④ 변수의 작명

camelcase //X

camelCase //O

의미 있는
이름으로
지어 주세요!



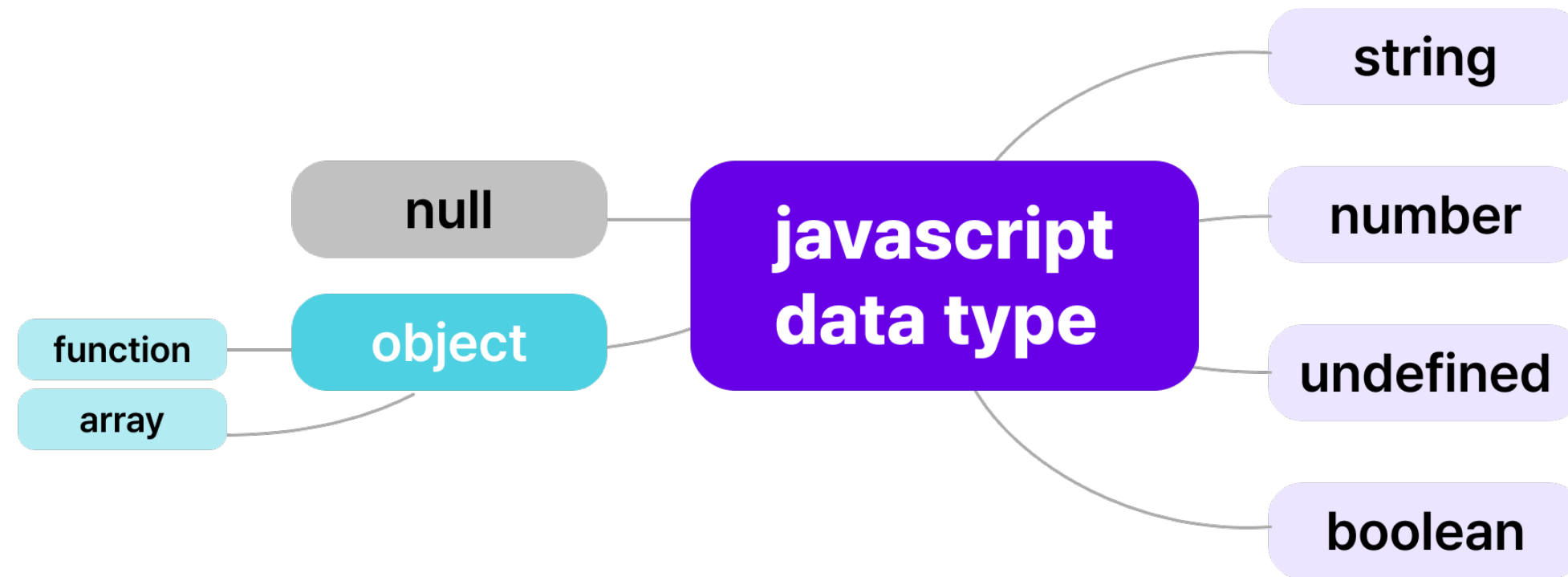
- 구체적이고, 연관성 있는 저장된 값을 잘 알 수 있는 이름이 좋습니다.
- 예약어를 사용하면 안됩니다.(if, var, const, let....)
- 변수의 단어가 두 개 이상일 시에는 어어 지는 첫 글자가 대문자로 시작되는 camel case를 사용하는 것이 좋습니다.
- 숫자로 시작하지 않습니다.
- 특수문자를 금지합니다. 예외 (_ , \$)



데이터타입-기본타입

Data type이란?

☑ Data type이란?



프로그램에서 다룰 수 있는 값의 종류

- 자바스크립트에서는 여러 가지 형태의 타입을 미리 정의하여 제공하고 있으며, 이것을 기본 타입이라고 합니다.
- 자바스크립트의 기본 타입은 크게 원시 타입(기본 타입-primitive type)과 객체 타입(object type)으로 구분할 수 있습니다.
- 객체를 제외한 모든 타입은 불변 값(변경할 수 없는 값)을 정의합니다. 예를 들어 (C 언어와는 달리) 문자열은 불변합니다. 이런 일련의 타입을 "원시 값"이라고 합니다.

☑ Number - 숫자



1,2,3,-1,0,1.2221

변수에 숫자를 입력할 때 사용

C언어의 경우, 정수냐 실수냐에 따라 int, float 등과 같이 다양한 숫자 타입이 존재합니다.
반면에 JavaScript는 느슨한 타입 체크 언어이므로 정수든 실수든 그냥 숫자 타입입니다.

④ Number - 숫자

```
var num1 =1; //num에 정수 1 저장  
var num2 = 2.8; //num에 소수 2.8 저장  
var num3 = 10e3; //num에 10의 3승을 저장  
  
console.log(typeof num1) //number 출력
```

JavaScript는 타입이 없기 때문에 숫자를 모두 64bit 부동소수점으로 표현합니다.
typeof 연산자를 이용해 데이터 타입을 출력해 봅니다.

📌 용어 해설

typeof 연산자

Data의 자료형을 확인하는 연산자입니다.

☑ string - 문자



`'문자열','123','0','text'`

변수에 문자를 입력할 때 사용

- 문자열(String) 타입은 텍스트 데이터를 나타내는데 사용합니다.
- 문자열은 0개 이상의 16bit 유니코드 문자(UTF-16) 들의 집합으로 대부분의 전세계의 문자를 표현할 수 있습니다.
- 문자열은 작은 따옴표(") 또는 큰 따옴표("") 안에 텍스트를 넣어 생성합니다.
- 가장 일반적인 표기법은 작은 따옴표를 사용합니다.

☑ string - 문자



```
const name1 = "elice"
const a ="나는 ";
const b=" 입니다."

console.log(a + name1 +b) //나는 elice 입니다.
const age = 20
const c=a + age + b  //나는 20 입니다.
console.log(typeof c) //string
```

- 자바스크립트에서는 숫자와 문자열을 더할 수도 있습니다.
- 이럴 경우에 자바스크립트는 숫자를 문자열로 자동 변환하여, 두 문자열을 연결하는 연산을 수행합니다.

☑ boolean - 논리



true, false

참과 거짓을 나타낼 때

- 불리언 타입의 값은 참과 거짓으로 구분되는 조건에 의해 프로그램의 흐름을 제어하는 조건문에서 자주 사용됩니다.
- 비어있는 문자열과 null, undefined, 숫자 0은 false로, 숫자 1은 true로 간주됩니다.

☑ boolean - 논리



```
let trueVariable = true;  
let falseVariable = false;  
console.log(trueVariable == falseVariable) // false 출력
```

- boolean 타입에 해당하는 값은 true, false 두 가지 밖에 없습니다.
- 이 값들을 '진리값'이라고 부릅니다. 프로그래밍에서의 진리값은 어떤 조건이 참인지 거짓인지를 나타내기 위해 사용됩니다.

☑ undefined

undefined

선언은 했으나, data가 담기지 않았을 때

- undefined 타입의 값은 undefined가 유일합니다.
- 선언 이후 값을 할당하지 않은 변수는 undefined 값을 가집니다.
- 즉, 선언은 되었지만 값을 할당하지 않은 변수에 접근하거나 존재하지 않는 객체 property에 접근할 경우 undefined가 반환됩니다.

📖 용어 해설

Property란?

Key와 value로 되어있는 object data의 한 쌍

☑ undefined

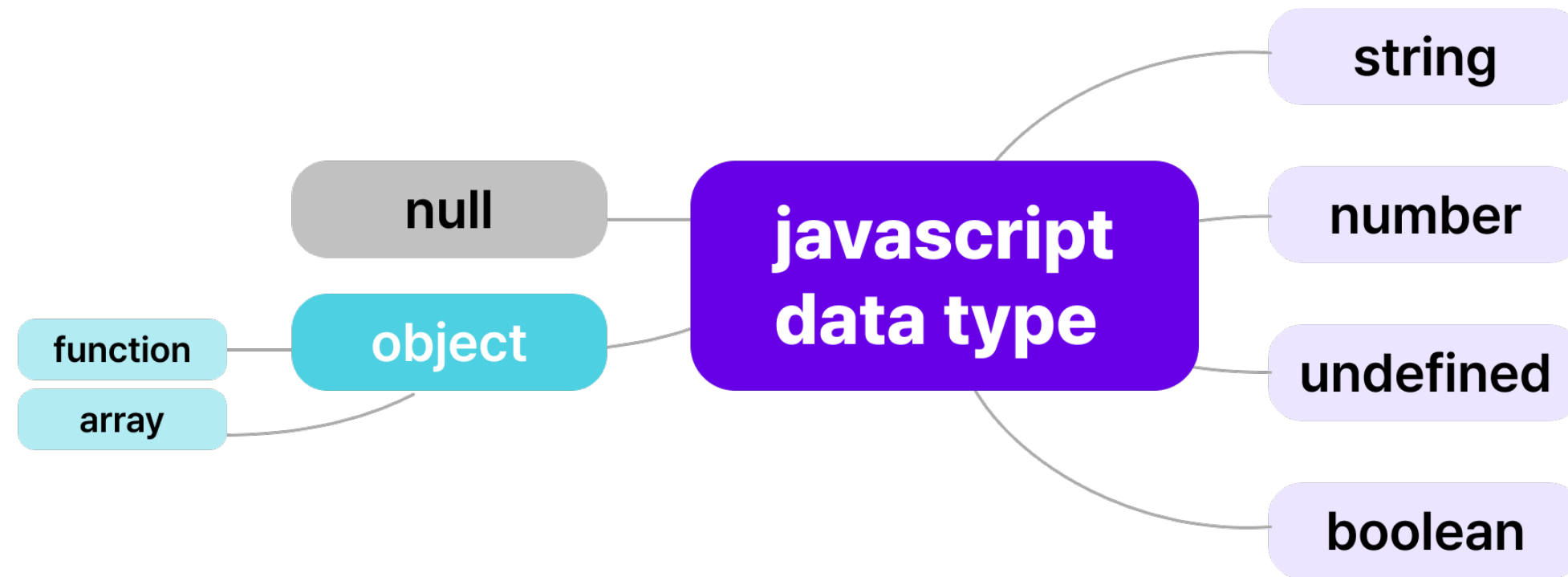


```
let a;  
console.log(a); // undefined  
console.log(typeof a); // undefined
```

- undefined 타입의 값은 undefined가 유일합니다.
- 선언 이후 값을 할당하지 않은 변수는 undefined 값을 가집니다.
- 즉, 선언은 되었지만 값을 할당하지 않은 변수에 접근하거나 존재하지 않는 객체 property에 접근할 경우 undefined가 반환됩니다.

데이터타입-객체타입

☑ Data type-객체타입



'자바스크립트는 기본타입을 제외한 모든 값은 객체다.'

- 자바스크립트의 객체는 '이름(key):값(value)'의 형태로 프로퍼티들을 저장하는 컨테이너입니다.
- 기본 타입으로 분류 되는 숫자, 문자열, 불린 값, undefined, null 값을 제외한 모든 값은 객체로 취급됩니다.

☑ object



```
{key : value}
```

다수의 property를 하나의 이름으로 묶어놓은 집합체

- 자바스크립트는 객체(object) 기반의 스크립트 언어로서 자바스크립트를 이루고 있는 거의 “모든 것”이 객체입니다.
- 객체의 선언 : `{}`
- 객체의 형태 : `이름(key):값(value)`
- Property : key와 value의 한 쌍을 뜻한다.

☑ object



```
var emptyObject = {};  
console.log(typeof emptyObject); // object  
var person = {  
  name: 'elice',  
  gender: 'male',  
  sayHello: function () { console.log('Hi! My name is ' + this.name); }  
};  
console.log(person.sayHello())//Hi! My name is Elice
```

- 자바스크립트의 객체는 **키(key)**와 **값(value)**으로 구성된 **프로퍼티(Property)**들의 집합이다.
- 프로퍼티의 값으로 자바스크립트에서 사용할 수 있는 모든 값을 사용할 수 있다.
- 자바스크립트의 함수는 일급 객체이므로 값으로 취급할 수 있다. 따라서 프로퍼티 값으로 함수를 사용할 수도 있으며 프로퍼티 값이 함수일 경우, 일반 함수와 구분하기 위해 **메소드**라 부른다.
- 메소드는 객체에 제한되어 있는 함수를 의미한다.

☑ array

```
[ 'text', 0, function() {}, Boolean, null ]
```

순서가 있는 데이터의 집합

- 배열은 리스트와 비슷한 객체로서 순회와 변형 작업을 수행하는 메소드를 갖습니다.
- JavaScript 배열은 길이도, 각 요소의 자료형도 고정되어 있지 않습니다.
- Object와 달리 배열은 인덱스가 0으로 시작하는 숫자형 인덱스를 사용하며 한 개의 변수의 여러 개의 값을 순차적으로 저장할 수 있습니다.

✍ 용어 해설

.length

인덱스로 배열 요소의 개수를 확인할 수 있다.

☑ array 생성



```
var myArray1 = new Array(); // 길이가 0인 예전 방식의 배열 생성
var myArray2 = new Array(10); // 길이가 10인 예전 방식의 배열 생성
var myArray3 = []; // 길이가 0인 literal notation 방식의 배열 생성
var myArray4 = new Array(1, 2, 3, "elice", "cat"); // 예전 방식의 생성과 동시에 초기화
var myArray5 = [1, 2, 3, "elice", "cat"]; // literal notation 방식의 생성과 동시에 초기화

console.log(myArray1);
```

- 배열 리터럴을 사용하는 것이 JavaScript 배열을 만드는 가장 쉬운 방법입니다.
- 배열은 단일 이름으로 많은 값을 보유할 수 있으며 색인 번호(index)를 참조하여 값에 액세스할 수 있습니다.

☑ array 생성



```
const array_name = ['item1', 'item2'];  
console.log(array_name);
```

```
const cars1 = [ "Saab", "Volvo", "BMW"];  
const cars2= [];  
cars[0]= "Saab";  
cars[1]= "Volvo";  
cars[2]= "BMW";
```

- 배열 리터럴을 사용하는 것이 JavaScript 배열을 만드는 가장 쉬운 방법입니다.
- 배열은 단일 이름으로 많은 값을 보유할 수 있으며 색인 번호를 참조하여 값에 액세스할 수 있습니다.
- const 키워드로 배열을 선언하는 것이 일반적입니다.

④ function



```
function 함수이름(매개변수1, 매개변수2,...) {  
    함수가 호출되었을 때 실행하고자 하는 실행문;  
}
```

일련의 작업을 목적으로 수행에 설계된 독립적 블록

- JavaScript의 함수는 객체 타입을 갖는 1급 객체입니다.
- 변수나, 데이터를 구조 안에 담을 수 있습니다.
- 다른 함수에 파라미터(매개변수)로 전달 할 수 있습니다.
- 리턴값을 사용할 수 있습니다.

☑ function



```
function addNum(x, y) {  
    return x + y;  
}  
console.log(addNum(2, 3))
```

- 자바스크립트에서는 함수도 하나의 타입(datatype)입니다.
- 따라서 함수를 변수에 대입하거나, 함수에 프로퍼티를 지정하는 것도 가능합니다.
- 또한, 자바스크립트 함수는 다른 함수 내에 중첩되어 정의될 수도 있습니다.
- 자바스크립트에서 함수는 반환(return)문을 포함할 수 있습니다.
- 이러한 반환문을 통해 호출자는 함수에서 실행된 결과를 전달받을 수 있습니다.

☑ 화살표 함수



```
function addNum(x, y) {  
  return x + y;  
}
```



```
const addNum = (x, y) => x + y;
```

ES6부터 새롭게 등장한 함수 작성 방식입니다. function 키워드 대신 => (화살표) 기호를 사용합니다. 같은 동작을 더 짧고 간결하게 표현할 수 있습니다.

- 코드가 짧고 가독성이 좋습니다.
- 한 줄짜리 함수는 return도 생략 가능합니다.

☑ null

null

명시적으로 비어 있음을 표시할 때

- 프로그래밍 언어에서 null은 의도적으로 변수에 값이 없다는 것을 명시할 때 사용합니다.
- 이는 변수가 기억하는 메모리 어드레스의 참조 정보를 제거하는 것을 의미하며 자바스크립트 엔진은 누구도 참조하지 않는 메모리 영역에 대해 가비지 컬렉션을 수행할 것입니다.

🖋 용어 해설

가비지 컬렉션

garbage collection
더 이상 쓰지 않는 메모리를 자동으로 정리해주는 기능

☑ null



```
var nullType = null;  
console.log(typeof null); // object
```

- **typeof** 에서 **null** 단, typeof로 자료형을 확인할 때 object(객체)를 반환하는데, null이 빈 참조를 나타내는 데 자주 사용되기 때문이다.
- 그래서 원시 자료형으로 생각해도 되지만 엄밀히 말해서는 null은 객체이고 참조 자료형이다.

Property와 Methods

☑ Property와 methods



```
{key : value}
```

```
{ key : function(){} }
```

property

method

- **프로퍼티**는 object의 한 쌍을 말합니다.
- 메소드는 object의 프로퍼티 값이 함수일 경우 일반적인 함수와 구별하기 위해 **메소드**라고 부릅니다.
- 자바스크립트에서 사용자를 위해 사전에 작성된 편의기능 메소드에 대해 알아보니다.

☑ 문자열 property method



method	설명
.split(" ")	공백 기준으로 문자를 나눈 후 배열로 출력
.concat('t')	기존 문자열에 추가하여 출력
.length	문자열 내의 문자 개수를 반환
.charAt(n)	문자열의 n번의 문자 출력
.indexOf('t')	문자열 't'의 순서값 출력

☑ 문자열 property method



```
const str1 = "hello elice";

console.log(str1.indexOf('e')); // 1
console.log(str1.length); // 11
console.log(str1.charAt(6)); // e
console.log(str1.concat(' bye!')) // hello elice bye!
console.log(str1.split(" ")) // ['hello', 'elice']
```


☑ 배열 property method



method	설명
.length	배열의 개수 출력
.push('t')	배열 맨 뒤에 삽입
.unshift('t')	배열의 맨 앞에 삽입
.pop()	배열 뒤의 데이터 제거
.splice(n,i)	배열의 n번째 인덱스부터 i개 제거

☑ 배열 property method



```
const arr = ['elice', 'cat', 'javascript'];  
console.log(arr[0]); // 'elice'  
  
console.log(arr.length); //3  
arr.push('moon') //마지막요소로 'moon'추가  
arr.unshift('hello')//첫번째 요소로 'hello'추가  
arr.pop()//마지막요소로 'moon' 제거  
arr.splice(1,2) //elice cat 제거  
console.log(arr);
```


✔ Math method



Math 객체는 수학에서 자주 사용하는 상수와 함수들을 미리 구현해 놓은 자바스크립트 표준 내장 객체입니다.

method	설명
Math.abs()	절댓값 반환
Math.random()	0보다 크거나 같고 1보다 작은 무작위 숫자(random number)를 반환
Math.round()	인수로 전달받은 값을 소수점 첫 번째 자리에서 반올림하여 그 결과 값을 반환합니다.
Math.floor()	전달받은 값과 같거나 작은 수 중에서 가장 큰 정수

☑ Math method



Math 객체는 수학에서 자주 사용하는 상수와 함수들을 미리 구현해 놓은 자바스크립트 표준 내장 객체입니다.

```
console.log(Math.abs(-25))//25
console.log(Math.random(10))//소수점 출력
console.log(Math.floor(Math.random()*10))//0-10까지의 난수
console.log(Math.round(10.12345))//10
```


☑ 명시적 타입 변환



자바스크립트에서는 묵시적 타입 변환을 많이 사용하지만, 명시적으로 타입을 변환할 방법도 제공한다. 명시적 타입 변환을 위해 자바스크립트가 제공하는 전역 함수는 다음과 같다.

method	설명
Number()	숫자형으로 변환
String()	문자형으로 변환
Boolean()	Boolean 형으로 변환
parseInt()	문자를 숫자로 변환

☑ 명시적 타입 변환



자바스크립트에서는 묵시적 타입 변환을 많이 사용하지만, 명시적으로 타입을 변환할 방법도 제공한다. 명시적 타입 변환을 위해 자바스크립트가 제공하는 전역 함수는 다음과 같다.

```
console.log(typeof(Number('10')))//number  
console.log(typeof(String(10)))//string  
console.log(typeof(Boolean('10')))//boolean  
console.log(typeof(parseInt('10')))//number
```